Task 1: Black-Box Testing for Calculator
Java Program:

```
import java.util.Scanner;

public class Main {

    static double add(double a, double b) {
        return a + b;
    }

    static double subtract(double a, double b) {
        return a - b;
    }

    static double divide(double a, double b) {
        if (b == 0) {
            throw new ArithmeticException("Division by zero is not allowed");
        }
        return a / b;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        double a = sc.nextDouble();
```

```
        System.out.print("Enter second number: ");
        double b = sc.nextDouble();

        System.out.println("Addition: " + add(a, b));
        System.out.println("Subtraction: " + subtract(a, b));

        try {
            System.out.println("Division: " + divide(a, b));
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        }

        sc.close();
    }
}
```

Task 2: White-Box Testing for a Function with If-Else and Loops

```
public class Main {

    static int countPositive(int[] numbers) {
        int count = 0;

        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] > 0) {
                count++;
```

```
        } else {
            System.out.println(
                numbers[i] + " is not positive"
            );
        }
    }

    return count;
}

public static void main(String[] args) {

    int[] test1 = {1, -2, 3};
    int[] test2 = {-1, 0};
    int[] test3 = {};

    System.out.println(
        "Test 1 - Positive count: " +
        countPositive(test1)
    );

    System.out.println(
        "Test 2 - Positive count: " +
        countPositive(test2)
    );

    System.out.println(
        "Test 3 - Positive count: " +
```

```
            countPositive(test3)
        );
    }
}
```

Task 3: SimpleMath – JUnit Testing

1. SimpleMath.java

```
public class SimpleMath {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }
}
```

2. SimpleMathTest.java

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;
```

```
public class SimpleMathTest {

    SimpleMath math = new SimpleMath();

    @Test
    public void testAddition() {
        assertEquals(10, math.add(5, 5));
    }

    @Test
    public void testSubtraction() {
        assertEquals(5, math.subtract(10, 5));
    }

    @Test
    public void testMultiplication() {
        assertEquals(20, math.multiply(4, 5));
    }

    @Test
    public void testNegativeNumbers() {
        assertEquals(-5, math.add(-2, -3));
    }

    @Test
    public void testZero() {
        assertEquals(0, math.add(0, 0));
    }
```

```
}
```

Task 4: Test String Utilities – Reverse String and Palindrome Check

1. StringUtils.java

```
public class StringUtils {

    public String reverse(String str) {
        return new StringBuilder(str).reverse().toString();
    }

    public boolean isPalindrome(String str) {
        return str.equals(reverse(str));
    }
}
```

2. StringUtilsTest.java

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertTrue;
import static org.junit.Assert.assertFalse;

public class StringUtilsTest {

    StringUtils utils = new StringUtils();

    @Test
    public void testReverse() {
```

```
        assertEquals("olleh", utils.reverse("hello"));
    }

    @Test
    public void testReverseSingleCharacter() {
        assertEquals("a", utils.reverse("a"));
    }

    @Test
    public void testPalindromeTrue() {
        assertTrue(utils.isPalindrome("madam"));
    }

    @Test
    public void testPalindromeFalse() {
        assertFalse(utils.isPalindrome("hello"));
    }

    @Test
    public void testEmptyString() {
        assertEquals("", utils.reverse(""));
        assertTrue(utils.isPalindrome(""));
    }
}
```

Task 5: Test Divide Method Using assertThrows()

1. Calculator.java

```
public class Calculator {

    public int divide(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException(
                "Cannot divide by zero"
            );
        }
        return a / b;
    }
}
```

2. CalculatorTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {

    Calculator calc = new Calculator();

    @Test
    public void testDivision() {
        assertEquals(5, calc.divide(10, 2));
    }

    @Test
    public void testDivisionByZero() {
```

```
        ArithmeticException exception = assertThrows(
            ArithmeticException.class,
            () -> calc.divide(10, 0)
        );

        assertEquals(
            "Cannot divide by zero",
            exception.getMessage()
        );
    }

    @Test
    public void testNegativeDivision() {
        assertEquals(-5, calc.divide(-10, 2));
    }

    @Test
    public void testZeroNumerator() {
        assertEquals(0, calc.divide(0, 5));
    }
}
```

Task 6: Validate Sorting Function Correctness

1. Sorter.java

```
import java.util.Arrays;

public class Sorter {
```

```
    public int[] sort(int[] arr) {
        int[] result = arr.clone();
        Arrays.sort(result);
        return result;
    }
}
```

2. SorterTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class SorterTest {

    Sorter sorter = new Sorter();

    @Test
    public void testNormalSorting() {
        assertArrayEquals(
            new int[]{1, 2, 3, 4, 5},
            sorter.sort(new int[]{5, 3, 1, 4, 2})
        );
    }

    @Test
    public void testEmptyArray() {
        assertArrayEquals(
            new int[]{},
```

```
        sorter.sort(new int[]{})
    );
}

@Test
public void testAlreadySorted() {
    assertArrayEquals(
        new int[]{1, 2, 3},
        sorter.sort(new int[]{1, 2, 3})
    );
}

@Test
public void testNegativeNumbers() {
    assertArrayEquals(
        new int[]{-5, -2, 0, 3},
        sorter.sort(new int[]{3, -2, 0, -5})
    );
}

@Test
public void testDuplicateNumbers() {
    assertArrayEquals(
        new int[]{1, 2, 2, 3},
        sorter.sort(new int[]{2, 3, 2, 1})
    );
}
```

```
    @Test
    public void testSingleElement() {
        assertArrayEquals(
            new int[]{7},
            sorter.sort(new int[]{7})
        );
    }
}
```

Task 7: List Management with Lifecycle

Program: ListManagementTest.java

```
import org.junit.jupiter.api.*;
import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

public class ListManagementTest {

    List<String> list;

    @BeforeEach
    void setUp() {
        list = new ArrayList<>();
        System.out.println("List initialized");
    }
```

```
    @Test
    void testAddElements() {
        list.add("Java");
        list.add("Python");

        assertEquals(2, list.size());
        assertTrue(list.contains("Java"));

        System.out.println("Elements added: " + list);
    }

    @AfterEach
    void clearList() {
        list.clear();

        assertTrue(list.isEmpty());
        System.out.println("List cleared");
    }
}
```

Task 8: Test Lifecycle Execution Order

Program: LifecycleTest.java

```
import org.junit.jupiter.api.*;

public class LifecycleTest {
```

```
@BeforeAll
static void beforeAll() {
    System.out.println("1. Before All");
}

@BeforeEach
void beforeEach() {
    System.out.println("2. Before Each");
}

@Test
void testOne() {
    System.out.println("3. Test One");
}

@Test
void testTwo() {
    System.out.println("3. Test Two");
}

@AfterEach
void afterEach() {
    System.out.println("4. After Each");
}

@AfterAll
static void afterAll() {
    System.out.println("5. After All");
```

```
    }
}
```

Task 9: Test Empty Input, Null Input, and Single Element

1. InputValidator.java

```
public class InputValidator {

    public int getLength(String input) {
        if (input == null) {
            throw new IllegalArgumentException(
                "Input cannot be null"
            );
        }

        return input.length();
    }
}
```

2. InputValidatorTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class InputValidatorTest {

    InputValidator validator = new InputValidator();

    @Test
```

```
    void testEmptyInput() {
        assertEquals(0, validator.getLength(""));
    }

    @Test
    void testNullInput() {
        assertThrows(
            IllegalArgumentException.class,
            () -> validator.getLength(null)
        );
    }

    @Test
    void testSingleCharacter() {
        assertEquals(1, validator.getLength("A"));
    }

    @Test
    void testNormalInput() {
        assertEquals(5, validator.getLength("Hello"));
    }
}
```

Task 10: Design Test Cases for a Login System

1. LoginSystem.java

```
public class LoginSystem {
```

```
    public boolean login(String username, String password) {

        if (username == null || password == null ||
            username.trim().isEmpty() ||
            password.trim().isEmpty()) {
            return false;
        }

        return username.equals("admin") &&
               password.equals("1234");
    }
}
```

2. LoginSystemTest.java

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class LoginSystemTest {

    LoginSystem system = new LoginSystem();

    @Test
    void testValidLogin() {
        assertTrue(system.login("admin", "1234"));
    }

    @Test
```

```
void testInvalidUsername() {
    assertFalse(system.login("user", "1234"));
}

@Test
void testInvalidPassword() {
    assertFalse(system.login("admin", "wrong"));
}

@Test
void testBothInvalid() {
    assertFalse(system.login("user", "wrong"));
}

@Test
void testBlankUsername() {
    assertFalse(system.login("", "1234"));
}

@Test
void testBlankPassword() {
    assertFalse(system.login("admin", ""));
}

@Test
void testBothBlank() {
    assertFalse(system.login("", ""));
}
```

```
    @Test
    void testNullUsername() {
        assertFalse(system.login(null, "1234"));
    }

    @Test
    void testNullPassword() {
        assertFalse(system.login("admin", null));
    }

    @Test
    void testWhitespaceUsername() {
        assertFalse(system.login("   ", "1234"));
    }
}
```